

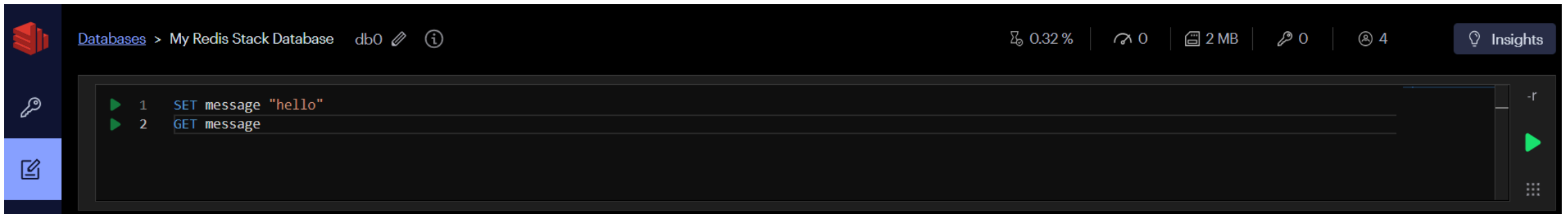
String 활용

1. Redis String

String은 Redis의 가장 기본적인 자료형입니다. 텍스트, 정수, 실수, JSON, 직렬화된 데이터 등을 저장할 수 있습니다. 값의 최대 크기는 크지만, 큰 값을 많이 저장하면 네트워크와 메모리 비용이 커지므로 캐시 대상을 적절히 나누어야 합니다.

message라는 Key에 "hello" 문자열 저장
SET message "hello"

message의 값 조회
GET message

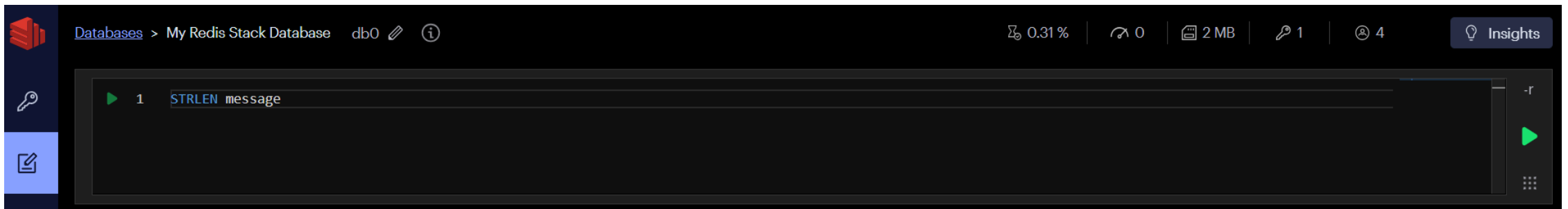


Databases > My Redis Stack Database db0 0.32 % 0 2 MB 0 4 Insights

```
1 SET message "hello"
2 GET message
```

Clear Results				
GET message	Jun 22, 15:05:28	2.877 msec		
"hello"				
SET message "hello"	Jun 22, 15:05:28	3.118 msec		
"OK"				

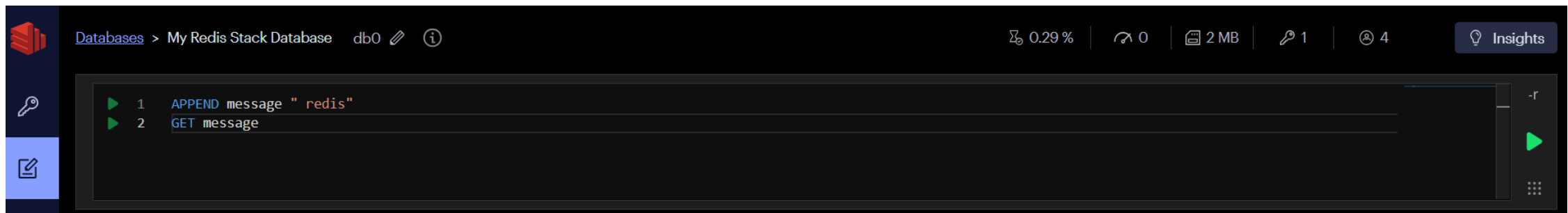
message 문자열의 길이(문자 수) 조회
STRLEN message



Clear Results				
STRLEN message	Jun 22, 15:06:19	0.415 msec		
(integer) 5				
GET message	Jun 22, 15:05:28	2.877 msec		

```
# message 문자열 끝에 " redis" 추가  
APPEND message " redis"
```

```
# 변경된 message 값 조회  
GET message
```



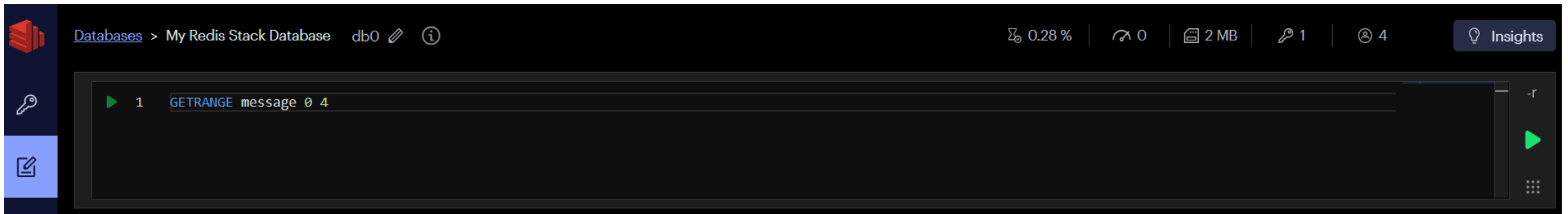
The screenshot shows a Redis Stack database interface. The top bar indicates the database is 'My Redis Stack Database' with 'db0' selected. The interface includes a sidebar with icons for databases, keys, and a command editor. The command editor contains two lines of code: '1 APPEND message " redis"' and '2 GET message'. The right side of the interface shows various metrics: 0.29% memory usage, 0 connections, 2 MB memory, 1 key, and 4 databases. An 'Insights' button is also visible.



The screenshot shows the results of the commands executed in the Redis Stack database. The results are displayed in a table with columns for the command, the result, the timestamp, and the execution time. The first command is 'GET message', which returned the result 'hello redis' at 15:07:17 on Jun 22, with an execution time of 1.114 msec. The second command is 'APPEND message " redis"', which returned the result '(integer) 11' at 15:07:17 on Jun 22, with an execution time of 1.329 msec. The interface includes a 'Clear Results' button and various icons for editing and navigating the results.

Command	Result	Timestamp	Execution Time
GET message	"hello redis"	Jun 22, 15:07:17	1.114 msec
APPEND message " redis"	(integer) 11	Jun 22, 15:07:17	1.329 msec

message 문자열의 일부만 조회 (0번부터 4번 인덱스까지)
GETRANGE message 0 4



Clear Results				
GETRANGE message 0 4	Jun 22, 15:08:05	1.337 msec	</> v	🗑️
"hello"				
GET message	Jun 22, 15:07:17	1.114 msec		🗑️

2. 여러 값 처리

```
MSET user:1:name "민수" user:2:name "지수" user:3:name "수진"  
MGET user:1:name user:2:name user:3:name
```

MSET 과 MGET 은 왕복 횟수를 줄일 수 있습니다.

다만 서로 관련 없는 많은 키를 무제한으로 한 번에 요청하면 서버와 네트워크에 부담이 되므로 적절한 크기로 나눕니다.



[Databases](#) > My Redis Stack Database db0  

0.29 % | 0 | 2 MB | 1 | 4  Insights



▶ 1 MSET user:1:name "민수" user:2:name "지수" user:3:name "수진"

▶ 2 MGET user:1:name user:2:name user:3:name

-r

▶

⋮

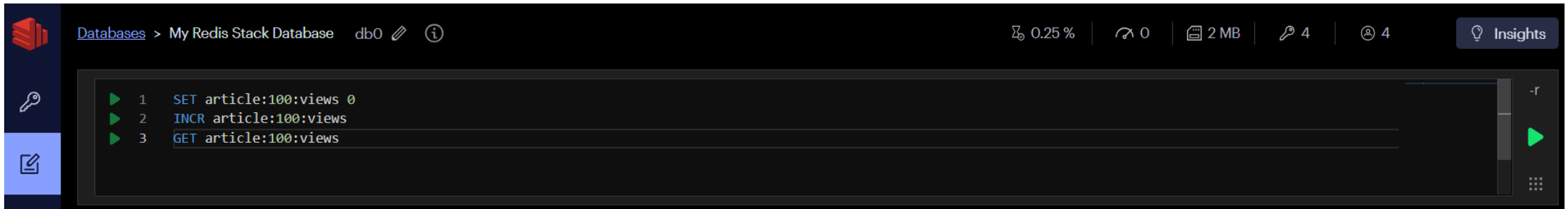
 Clear Results

MGET user:1:name user:2:name user:3:name	Jun 22, 15:08:58	1.44 msec				
1) "\xeb\xaf\xbc\x88\x98"						
2) "\xec\xa7\x80\x88\x98"						
3) "\xec\x88\x98\xa7\x84"						
MSET user:1:name "민수" user:2:name "지수" user:3:name "수진"	Jun 22, 15:08:58	1.035 msec				
"OK"						

3. 카운터

- INCR : 값을 1 증가

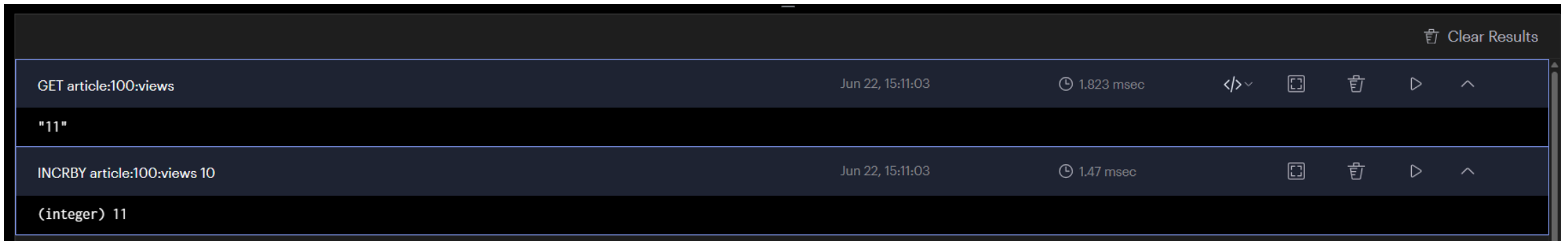
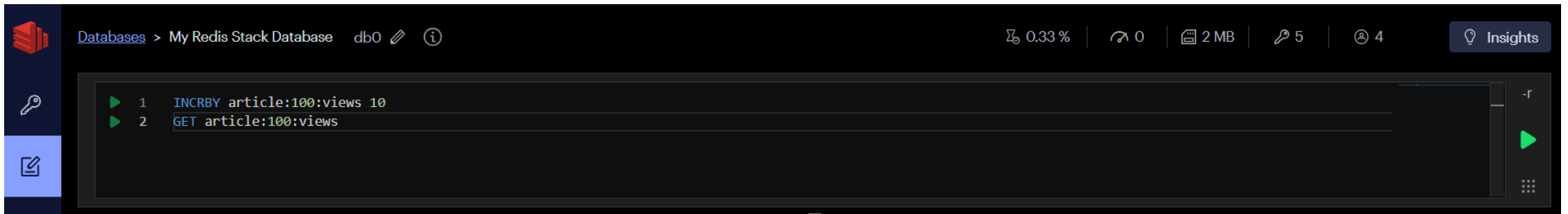
```
SET article:100:views 0  
INCR article:100:views  
GET article:100:views
```



					Clear Results
GET article:100:views	Jun 22, 15:10:13	3.031 msec	</>	📄	🗑️
"1"					
INCR article:100:views	Jun 22, 15:10:13	3.283 msec		📄	🗑️
(integer) 1					
SET article:100:views 0	Jun 22, 15:10:13	2.517 msec		📄	🗑️
"OK"					

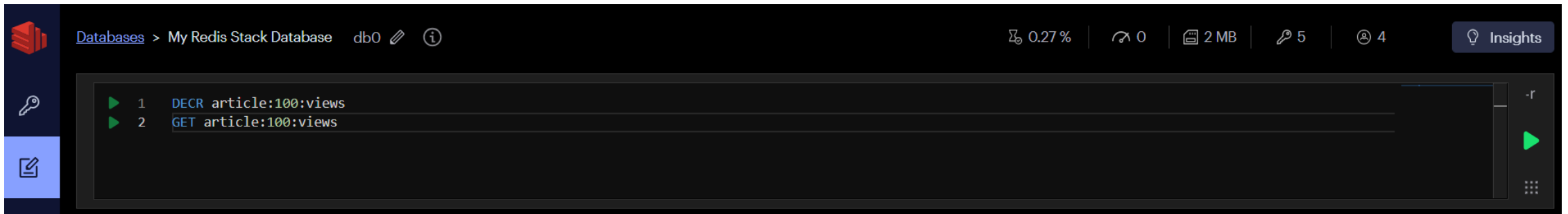
- INCRBY : 지정한 정수만큼 증가
- INCRBYFLOAT : 실수만큼 증가

```
INCRBY article:100:views 10  
GET article:100:views
```



- `DECR` , `DECRBY` : 값을 감소

```
DECR article:100:views  
GET article:100:views
```



					Clear Results
GET article:100:views	Jun 22, 15:12:04	1.642 msec	</>	📄	🗑️
"10"					
DECR article:100:views	Jun 22, 15:12:04	0.845 msec		📄	🗑️
(integer) 10					

4. JSON 저장

```
SET cache:product:1001 '{"id":1001,"name":"키보드","price":49000}' EX 60  
GET cache:product:1001
```

JSON 문자열은 객체 전체를 한 번에 저장하고 읽기 쉽지만, 일부 필드만 수정하려면 전체 값을 읽어 다시 저장해야 합니다. 필드 단위 조회와 수정이 필요하면 Hash를 고려합니다.



Databases > My Redis Stack Database db0  

0.33 % | 0 | 2 MB | 5 | 4  Insights

 1 SET cache:product:1001 '{"id":1001,"name":"키보드","price":49000}' EX 60

 2 GET cache:product:1001



 Clear Results

GET cache:product:1001Jun 22, 15:13:021.549 msec{ }~

{
 "id": 1001,
 "name": "키보드",
 "price": 49000
}

SET cache:product:1001 '{"id":1001,"name":"키보드","price":49000}' EX 60Jun 22, 15:13:021.157 msec

"OK"